

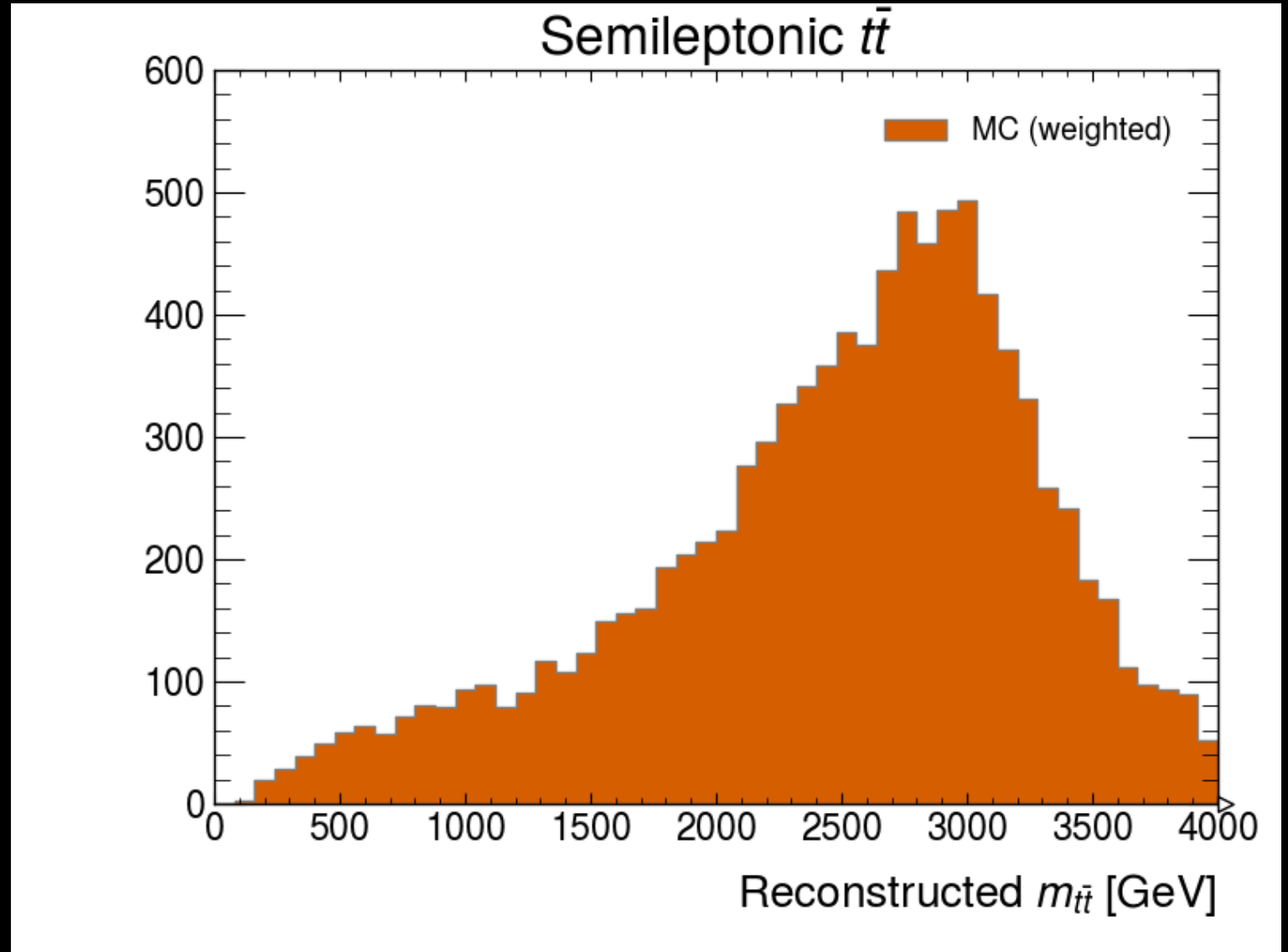
# From Query to Plot: Implementing a Tool-Based LLM Framework for ATLAS Analysis

G. Watts (UW/Seattle)

CHEP 2026

2026-05-28

On behalf of the ATLAS Collaboration



“Plot the  $p_T$  of all jets with  $|\eta| < 1$  in the rucio data set <rucio-did>”

“Plot **pT of the trijet system** with the mass closest to 172.5 GeV in each event and plot the **maximum b-tagging discriminant** value among the jets in the triplet in the rucio data set <rucio-dataset>.”

# Can We Make This “realistic”

What do **agents** need to:

1. Turn that into actual python code
2. Access experiment resources
  - GRID data!
3. Produce the plot

# What Infrastructure Do We Need?

## Context & Grounding

- Limit an LLM's options and force it to write working code
- Pick a technology stack
- Describe the vagaries of how to access ATLAS data

## An Agent Harness

- Code to feed our context & grounding to the LLM
- Code to run the resulting code
- Deal with experiment authentication, etc.

# Context & Grounding



ServiceX

Get data (ATLAS) off the GRID and convert it  
to a columnar format in a scalable way.  
(GRID Data/Skimming Access via REST API)

**Awkward  
Array**

Manipulate columns, filter, 4-vector algebra, etc.  
“numpy” for jagged data

**VECTOR**

Histogram manipulation, plotting (matplotlib),  
HEP formatting, etc.





Get data (ATLAS) off the GRID and convert it to a columnar format in a scalable way.   
(GRID Data/Skimming Access via REST API)



### **RDataFrame**

Manipulate columns, filter, 4-vector algebra, etc.

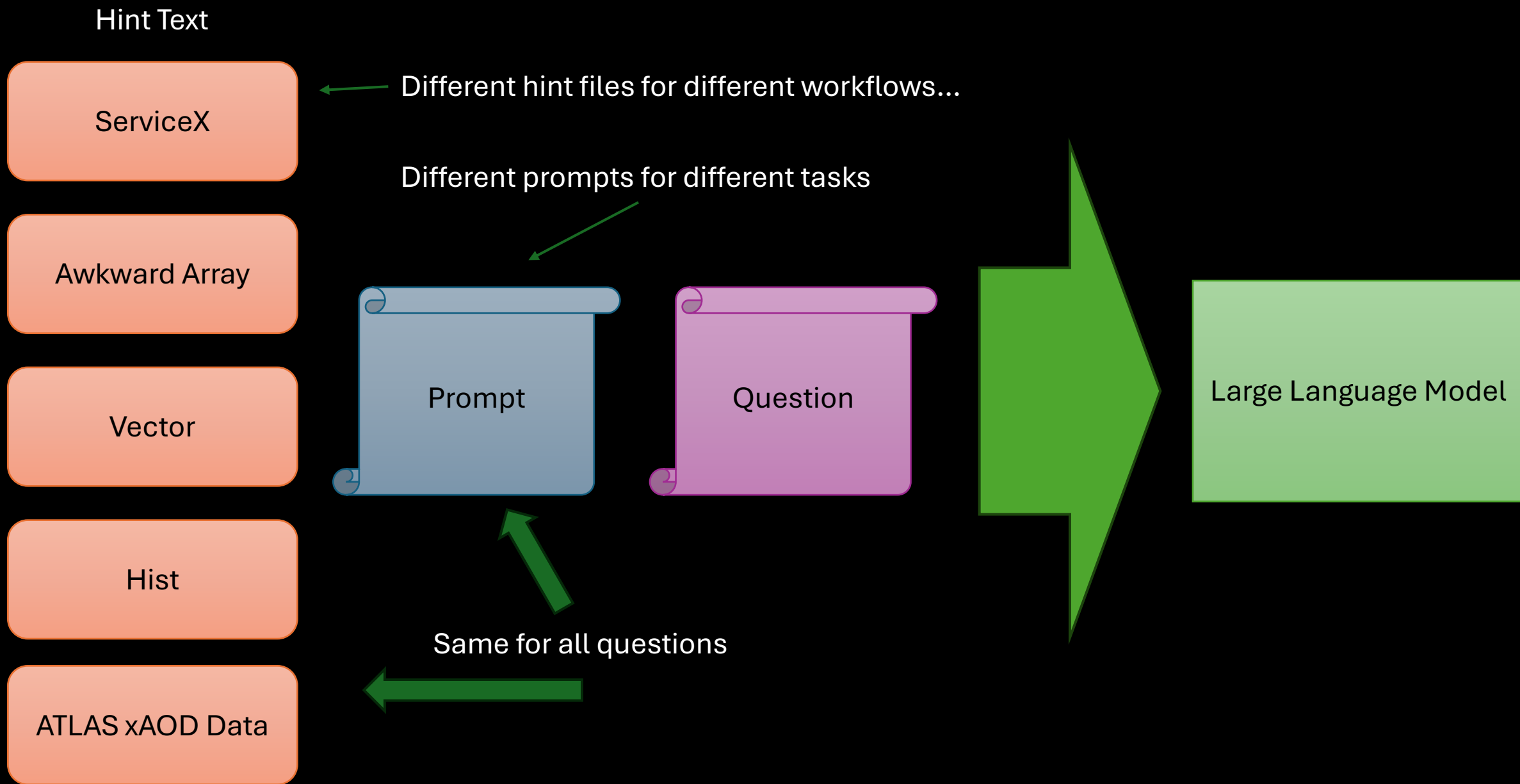
Histogram manipulation, plotting (matplotlib),  
HEP formatting, etc.

Workflow



Because we get quick, easy, immediate access to experiment GRID data.

- Requires a GRID dataset
- Requires a API key for GRID access



## Awkward Array Hints File

### Flattening Nested Arrays

Use `ak.flatten` to reduce nested list structure. By default, `ak.flatten(array)` removes one level of nesting (flattens along the `axis=1`). Setting `axis=None` will completely flatten an array, erasing all nesting (turning it into 1D):

```
import awkward as ak

array = ak.Array([[0, 1, 2], [], [3, 4], [5, 6, 7]])
flat_level1 = ak.flatten(array, axis=1) # Flatten one level (axis=1 by default):contentReference[oaicite:2]{index=2}
flat_all = ak.flatten(array, axis=None) # Flatten all levels into 1D:contentReference[oaicite:3]{index=3}

print(flat_level1) # [0, 1, 2, 3, 4, 5, 6, 7]
print(flat_all)    # [0, 1, 2, 3, 4, 5, 6, 7] (same here because only one level)
```

#### Notes:

- Always explicitly set the `axis` in the arguments to `flatten`. This will force us to think about what axis we are flattening and if the data has that access.
- If a variable has no array structure or is a 1D array, then `ak.flatten` will cause an error. Be sure your data has the requested `axis` if you are going to flatten.

### Combining Multiple Fields into Records

Use `ak.zip` to join parallel arrays into an array of records (structured data). The `ak.zip` function can combine arrays of equal length into one record per entry. For example, zipping an age array and a name array yields an array of records with fields `"age"` and `"name"`:

```
import awkward as ak

ages = ak.Array([18, 32, 41])
names = ak.Array(["Dorit", "Caitlin", "Bryn"])
people = ak.zip({"age": ages, "name": names}) # Combine fields into records:contentReference[oaicite:5]{index=5}
```

## ATLAS xAOD Data Model

### xAOD Event Data Model Hints

Some hints to help with accessing xAOD objects.

Important note about units:

- All momentum, energy, and mass units are in MeV (e.g. `pt`, `E`, `m`). Unless asked otherwise, convert them to GeV by dividing by 1000 as early as possible.
- All distance units are in millimeters. Convert them to meters as early as possible.

### Jets, Muons, Taus, Photons, Electrons

It is possible to access all these objects from the top event level, e.g.,

```
query = FuncADLQueryPHYS() \
    .Select(lambda e: e.Jets()) \
    .Select(lambda jets: {
        'pt': jets.Select(lambda j: j.pt()/1000.0),
        'eta': jets.Select(lambda j: j.eta()/1000.0),
    })
```

The only odd one are tau's, which use `e.TauJets("AnalysisTauJets")`.

These objects all have `pt`, `eta`, `phi` (with methods by those names). To access `px`, `py`, `pz` you have to get the 4-vector first:

```
query = FuncADLQueryPHYS() \
    .Select(lambda e: e.Jets()) \
    .Select(lambda jets: {
        'pt': jets.Select(lambda j: j.p4().px()/1000.0),
    })
```

## Awkward Array Hints File

## ATLAS xAOD Data Model

## Flattening Nested Arrays

Use `ak.flatten` to reduce nested list structure. By default, `ak.flatten(array)` removes one level of nesting (flattens along the `axis=1`). Setting `axis=None` will completely flatten an array, erasing all nesting (turning it into 1D):

```
import awkward as ak

array = ak.Array([[0, 1, 2], [], [3, 4], [5, 6, 7]])
flat_level1 = ak.flatten(array, axis=1) # Flatten one level
flat_all = ak.flatten(array, axis=None) # Flatten all levels

print(flat_level1) # [0, 1, 2, 3, 4, 5, 6, 7]
print(flat_all)    # [0, 1, 2, 3, 4, 5, 6, 7] (same here because)
```

## Notes:

- Always explicitly set the `axis` in the arguments to `flatten`. This ensures consistent behavior across different versions of Awkward Array.
- If a variable has no array structure or is a 1D array, then `ak.flatten` will not change it.

## Combining Multiple Fields into Records

Use `ak.zip` to join parallel arrays into an array of records (structured arrays). For example, zipping an age array and a name array:

```
import awkward as ak

ages = ak.Array([18, 32, 41])
names = ak.Array(["Dorit", "Caitlin", "Bryn"])
people = ak.zip({"age": ages, "name": names}) # Combine fields into records
```

## xAOD Event Data Model Hints

Some hints to help with accessing xAOD objects.

## Writing Hint Files

- Used a LLM to draft many of the bigger ones first (“deep research”)
  - Some evidence these are too wordy
- Wrote some by hand (e.g. xAOD Data Model) – no good public source of documentation
- Iterate – when the LLM gave a confusing answer, could often trace it to incorrect or unclear hint files! [Found some errors in source documentation]

These hint files end up being quite useful for humans – they can be written in a very condensed way!

You will be asked to write python code that will access ATLAS data, extract some items, and then make one or more plots.

If the user's prompt doesn't fit that request, please complain. **Generate** code using ServiceX to fetch the data, Awkward array to **manipulate** the data that comes back (with vector for doing any physics calculations like invariant mass), and Hist to **generate and plot** the histogram. **Write** the histogram/plots to `png` files.

If hints contain instructions to tell the user some bit of information, make sure to do that. ***If you are manipulating from ServiceX, use only Awkward array - do not use python lists, etc.*** Just give the user the code, do ***not try to create a pull request in any repo.***

**\*\* Hints \*\***

{hints}

**\*\* User Question \*\***

{question}

**\*\* Final Instructions \*\***

Please reply in the following format:

>>start-reply<<

<text describing what you have done as a bulleted list>

```python

<code>

```

>>end-reply<<

Added to clean up the output!

Modern LLM's are fine-tuned to explain – if you don't give them an outlet, they will just write it anyway!

All you actually want...

Prompt

"Simple" prompt in github

# An Agent Harness

# What Kind Of Harness?

## Web Page Interface

- Involves cut/paste
- Testing and Validation is difficult

## Command Line/App

- Codex CLI, Copilot CLI, Claude Code, OpenCode, etc.
  - And their partner app's
- Distribute prompts with prompt files, agent files, skill packages, etc.
  - Standards developing
- Under very rapid iteration
- Built in sandboxing
- The Modern Choice

## Roll Your Own

- Write in python
- Can specifically control flow
- Sandbox by docker
- Was all that was available in June 2025

# What Kind Of Harness?

## Web Page Interface

- Involves cut/paste
- Testing and Validation is difficult

## Full Agentic Harness: Command Line/App

- Codex CLI, Copilot CLI, Claude Code, OpenCode, etc.
  - And their partner app's
- Distribute prompts with prompt files, agent files, skill packages, etc.
  - Standards developing
- Under very rapid iteration
- Built in sandboxing
- The Modern Choice

## Roll Your Own

- Write in python
- Can specifically control flow
- Sandbox by docker
- Was all that was available in June 2025

# What Kind Of Harness?

## Web Page Interface

- Involves cut/paste
- Testing and Validation is difficult

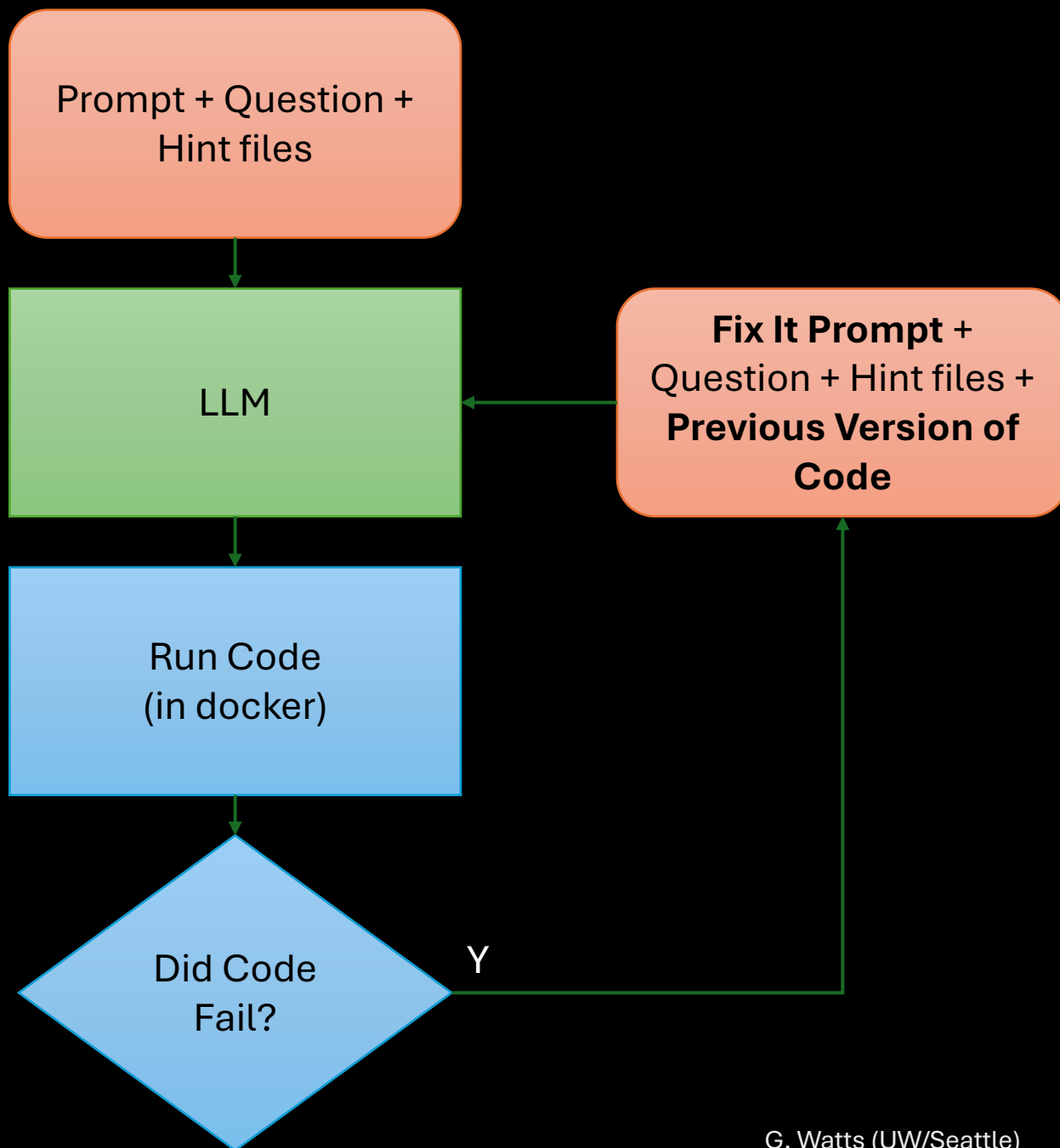
## Full Agentic Harness: Command Line/App

- Codex CLI, Copilot CLI, Claude Code, OpenCode, etc.
  - And their partner app's
- Distribute prompts with prompt files, agent files, skill packages, etc.
  - Standards developing
- Under very rapid iteration
- Built in sandboxing
- The Modern Choice

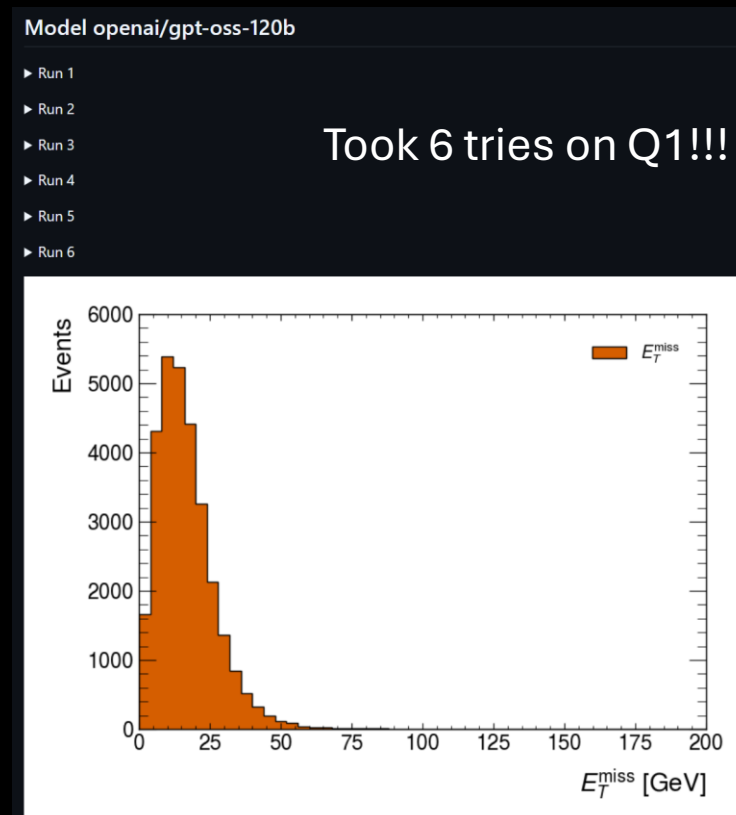


## Roll Your Own

- Write in python
- Can specifically control flow
- Sandbox by docker
- Was all that was available in June 2025



- Fix It Prompt is very similar to first prompt
  - Includes old code
  - stderr, stdout
- Iterations are allowed up to some limit



# Handling Crashes

## Fix in Prompt Files:

- Obscure failure mode in ServiceX
- Misuse of awkward array

## Fix in Harness:

- Timeout due to network issues (e.g. retry)
- Out of tokens (crash and wait for user to pay up)

stdout:

```
etmiss: Transform _____ 0/0 --:--
      Download _____ 0/0 --:--
```

stderr:

```
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
/work/.venv/lib/python3.13/site-packages/func_adl_servicex_xaodr25/trigger.py:127: SyntaxWarning: invalid escape sequence '\D'
  object. Close match is done as a function of $\Delta R < `dr`$.
Traceback (most recent call last):
  File "/app/script.py", line 36, in <module>
    result = to_awk(
              deliver(
...<10 lines>...
            )
          )
  File "/work/.venv/lib/python3.13/site-packages/servicex_analysis_utils/materialization.py", line 57, in to_awk
    raise ValueError(f"ServiceX result path list for {sample} cannot be empty.")
```

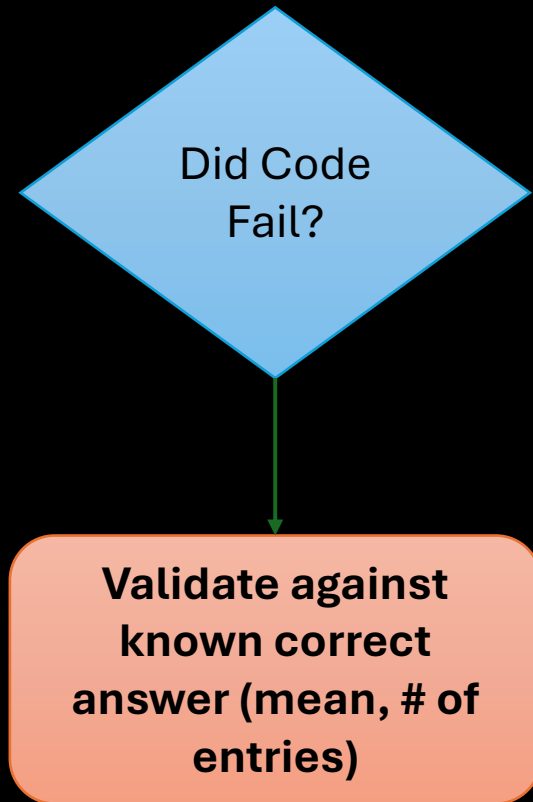
This is a crash in ServiceX

- No data was returned
- Note the “0/0” – no files were found for ServiceX to process.
- This means that the dataset name is likely bad.

BUG:

- Note there is no informative error message...
- I had to use my expert knowledge of the system to realize what was going on.
- **Just add this to the hint file and now the LLM is also an expert!**

# Validation



## Analysis Hallucinations

1. The LLM can't produce a plot at all
  - Mostly this is a LLM that struggles with a complex questions
2. Produces an incorrect plot
  - Likely would be given to a human to identify

All test questions have **human-checked correct answers**

- Harness checks the plot once it has been successfully produced
- LLM has no chance to fix it if it fails validation

# Performance

# What Plots?



## Benchmark description

## Motivation

|                                                                                                                                                                                                                                                       |                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Plot the missing $E_T$ of all events.                                                                                                                                                                                                                 | Loop over events and get an event-level variable.                                                                                                               |
| Plot $p_T$ of all jets in all events.                                                                                                                                                                                                                 | Loop over an array in each event.                                                                                                                               |
| Plot $p_T$ of jets with $ \eta  < 1$ .                                                                                                                                                                                                                | Loop over an array that is filtered.                                                                                                                            |
| Plot the missing $E_T$ of events that have at least two jets with $p_T > 40$ GeV.                                                                                                                                                                     | Loop over an array and aggregate the results to filter at the event level.                                                                                      |
| Plot the missing $E_T$ of events that have an opposite-sign muon pair with an invariant mass between 60 and 120 GeV.                                                                                                                                  | Loop on pairs of objects in one collection and do four-vector algebra.                                                                                          |
| Plot $p_T$ of the trijet system with the mass closest to 172.5 GeV in each event and plot the maximum $b$ -tagging discriminant value among the jets in the triplet.                                                                                  | Loop over combinations of objects in the same collection and extract a property of the combinations other than the key used to sort them.                       |
| Plot the sum of $p_T$ of jets with $p_T > 30$ GeV that are not within 0.4 in $\Delta R$ of any lepton with $p_T > 10$ GeV.                                                                                                                            | Loop over two different collections.                                                                                                                            |
| For events with at least three leptons and a same-flavor opposite-sign lepton pair, find the same-flavor opposite-sign lepton pair with the mass closest to 91.2 GeV and plot the transverse mass of the missing energy and the leading other lepton. | Perform a task for which the formulation in an imperative language is easy but the translation to a functional query language may be less clear or inefficient. |

G. Watts (UW/Seattle)

## HEP Data Query Challenges

Mason Proffitt, Emma Torró, Gordon Watts  
University of Washington, Seattle



### Motivation

- Several different code interfaces exist for analyzing HEP data, and new ones are being created
- Each of these analysis description languages (ADLs) needs to be capable of performing common analysis tasks
- The most accessible documentation of an ADL is by working examples inspired by real analyses
- Fair comparison of ADLs requires code samples of each performing the same tasks on the same input

### Index repository

- Ideas for common analysis tasks to use as reference were discussed in the HSF Data Analysis Working Group
- An initial list of data query challenges or *functionality benchmarks* was established
- A new GitHub repository was created to document the functionality benchmarks
- Contributors have implemented solutions to the challenges with different ADLs
- The new repository provides an index to these ADLs and links to the implementations of the benchmark tasks

### Language catalog

- The functionality benchmarks have been implemented in the following languages so far:

| Analysis description language                             | Programming language | Description                                                      |
|-----------------------------------------------------------|----------------------|------------------------------------------------------------------|
| RDataFrame                                                | C++                  | High-level interface within ROOT for declarative analysis        |
| NAIL (Natural Analysis Implementation Language)           | Python               | Interface for declarative analysis using RDataFrame as a backend |
| groot                                                     | Go                   | Pure Go package that provides read/write access to ROOT files    |
| coffea (Columnar Object Framework For Effective Analysis) | Python               | Builds on numpy and awkward-array for columnar data analysis     |

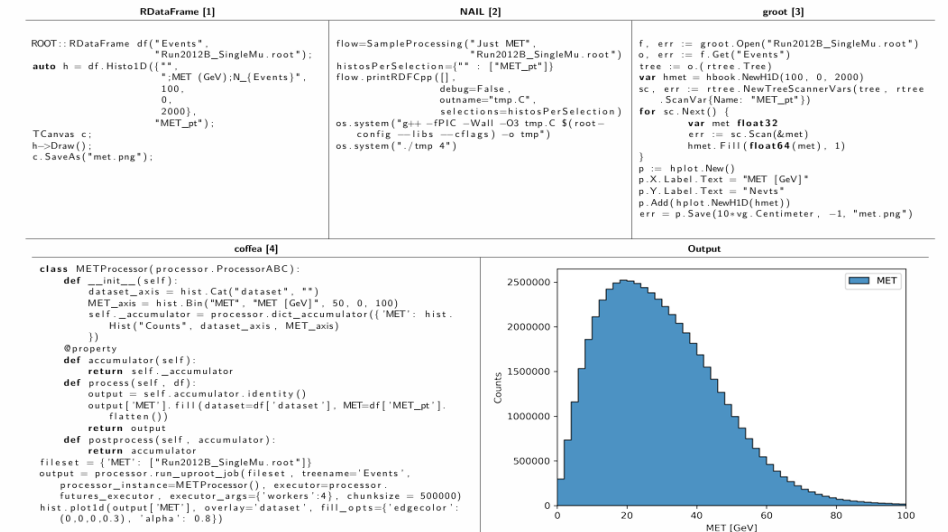
### Functionality benchmarks

- The current list of functionality benchmarks is shown in the table below
- The right column explains the generic type of functionality being exemplified
- This will be expanded as new challenges are suggested as GitHub issues to cover missing analysis use cases

| Benchmark description                                                                                                                                                                                                                                 | Motivation                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Plot the missing $E_T$ of all events.                                                                                                                                                                                                                 | Loop over events and get an event-level variable.                                                                                                               |
| Plot $p_T$ of all jets in all events.                                                                                                                                                                                                                 | Loop over an array in each event.                                                                                                                               |
| Plot $p_T$ of jets with $ \eta  < 1$ .                                                                                                                                                                                                                | Loop over an array that is filtered.                                                                                                                            |
| Plot the missing $E_T$ of events that have at least two jets with $p_T > 40$ GeV.                                                                                                                                                                     | Loop over an array and aggregate the results to filter at the event level.                                                                                      |
| Plot the missing $E_T$ of events that have an opposite-sign muon pair with an invariant mass between 60 and 120 GeV.                                                                                                                                  | Loop on pairs of objects in one collection and do four-vector algebra.                                                                                          |
| Plot $p_T$ of the trijet system with the mass closest to 172.5 GeV in each event and plot the maximum $b$ -tagging discriminant value among the jets in the triplet.                                                                                  | Loop over combinations of objects in the same collection and extract a property of the combinations other than the key used to sort them.                       |
| Plot the sum of $p_T$ of jets with $p_T > 30$ GeV that are not within 0.4 in $\Delta R$ of any lepton with $p_T > 10$ GeV.                                                                                                                            | Loop over two different collections.                                                                                                                            |
| For events with at least three leptons and a same-flavor opposite-sign lepton pair, find the same-flavor opposite-sign lepton pair with the mass closest to 91.2 GeV and plot the transverse mass of the missing energy and the leading other lepton. | Perform a task for which the formulation in an imperative language is easy but the translation to a functional query language may be less clear or inefficient. |

### Challenge implementation examples

- An implementation of the first functionality benchmark for each ADL is illustrated below
- All implementations produce a histogram similar to the one at the bottom-right



### Summary

- A list of functionality benchmarks designed to demonstrate how to perform common analysis tasks in an analysis description language has been created
- Several contributors have written implementations of these benchmark tasks in different analysis languages
- A repository documenting the functionality benchmarks, as well as indexing analysis languages and their implementations of the benchmarks has been created at <https://github.com/iris-hep/adl-benchmarks-index>

### References

### Acknowledgements



This material is based upon work supported by the National Science Foundation under Cooperative Agreement OAC-1836650. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

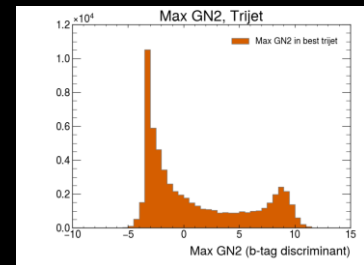
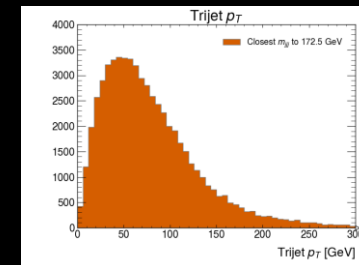
M. Proffitt – CHEP 2019

# What Does The LLM Have To Think Through?

**Question 6:** For events with at least three jets, plot the  $p_T$  of the trijet four-momentum that has the invariant mass closest to 172.5 GeV in each event and plot the maximum b-tagging discriminant value among the jets in this trijet

1. **Loop** over all the events
2. **Fetch** the jets from the event if it has at least 3 jets
3. **Form** all 3-pair combinations
4. **Pick** the 3-pair combination closest to mass 172.5 GeV (top mass)
5. **Fill** an appropriately defined histogram
6. **Fetch** the b-tagging discriminate for all three jets
7. **Fill** an appropriately defined histogram with the largest discriminate value.
8. **Plot** the histograms

*“Loop over combinations of objects in the same collection and extract a property of the combinations other than the key used to sort them.”*



## Question 1: Plot the $ET_{\text{miss}}$ of all events

- Used ServiceX FuncADL (PHYSLITE) to fetch only the event MissingET magnitude (converted to GeV) to minimize data movement.
- Queried the specified rucio dataset and delivered NFiles=1 (kept small for a quick example; you can increase NFiles yourself when ready).
- Converted the ServiceX result to an Awkward Array and filled a Hist histogram (50 bins, 0–200 GeV as a reasonable MET range).
- Plotted with mplhep ATLAS style and saved to etmiss.png.

(notes from LLM response)

```
import awkward as ak
from servicex import deliver, ServiceXSpec, Sample, dataset
from servicex_analysis_utils import to_awk
from func_adl_servicex_xaodr25 import FuncADLQueryPHYSLITE

from hist import Hist
import matplotlib.pyplot as plt
import mplhep as hep

# Use ATLAS plotting style
plt.style.use(hep.style.ATLAS)

# Build the ServiceX query: get MissingET (first object), then select its magnitude in GeV
base_query = FuncADLQueryPHYSLITE()
met_query = (
    base_query
    .Select(lambda e: e.MissingET().First())
    .Select(lambda m: {"met": m.met() / 1000.0}) # Convert to GeV
)

# Dataset name (from user)
ds_name = "mc23_13p6TeV:mc23_13p6TeV.801167.Py8EG_A14NNPDF23LO_jj_JZ2.deriv.DA0D_PHYSLITE.e8514_e8528_a911_s4114_r15224_r15225_1"

# Deliver the data (use NFiles=1 for a quick test run)
results = to_awk(
    deliver(
        ServiceXSpec(
            sample=[
                Sample(
                    Name="etmiss_fetch",
                    Dataset=dataset.Rucio(ds_name),
                    NFiles=1,
                    Query=met_query
                )
            ]
        )
    )

# Extract the MET array (Awkward)
met = results["etmiss_fetch"].met # units: GeV

# Build histogram: 50 bins from 0 to 200 GeV
h = (
    Hist.new
    .Reg(50, 0.0, 200.0, name="met", label=r"$E_T^{\mathrm{miss}}$ [GeV]")
    .Int64()
)

# Fill histogram (Hist accepts array-like; Awkward array works or can use ak.to_numpy(met))
h.fill(met=met)

# Plot and save
fig, ax = plt.subplots()
h.plot(histtype="fill", linewidth=1, edgecolor="gray", label=r"All events")
ax.legend()
ax.set_xlabel(r"$E_T^{\mathrm{miss}}$ [GeV]")
ax.set_title("Missing $E_T$")
fig.savefig("etmiss.png")
plt.close(fig)
```

## Question 6: For events with at least three jets, plot the $p_T$ of the trijet four-momentum that has the invariant mass closest to 172.5 GeV in each event and plot the maximum b-tagging discriminant value among the jets in this trijet

- Uses ServiceX (FuncADL R25 PHYSLITE) to fetch, in one delivery (NFiles=1), only what's needed: per-event jet kinematics (pt, eta, phi, mass in GeV) and the b-tagging discriminant (**GN2v01** tagger weight) via the BTaggingSelectionTool (FixedCutBEff\_77 operating point).
- Filters events at the ServiceX level to require at least three jets, reducing data moved.
- Builds Awkward Momentum4D vectors, forms all trijet combinations per event, selects the combination with invariant mass closest to 172.5 GeV, then:
  - Plots the  $p_T$  of the selected trijet four-momentum.
  - Plots the maximum b-tagging discriminant among the three jets in the selected trijet.
- Uses Awkward + Vector for all physics calculations and Hist for histogramming; saves plots to PNG files.
- Dataset: mc23\_13p6TeV:mc23\_13p6TeV.601237.PhPy8EG\_A14\_ttbar\_hdamp258p75\_allhad.deriv.DAOD\_PHYSLITE.e8514\_s4369\_r16083\_p6697.
- Note: Only one file is processed (NFiles=1) here; when you are ready, you can scale up yourself.

Boilerplate to deal with b-tagging

Filtering, combinatorics, plots

```
import awkward as ak
import vector
import numpy as np
from hist import Hist
import matplotlib.pyplot as plt
import uproot as hep

from service import deliver, ServiceSpec, Sample, dataset
from service_analysis_utils import import_to_awk
from func_adl_services_xadrs import FuncADLQueryPHYSLITE

# ..... XAD Tool Access helpers (must be included verbatim) .....
import ast
from dataclasses import dataclass
from typing import Callable, Iterable, List, Optional, Tuple, TypeVar
from func_adl import ObjectStream
from func_adl import FuncADLCallable

T = TypeVar("T")

@dataclass
class ToolInfo:
    name: str

def make_a_tool(
    query: ObjectStream[T],
    tool_name: str,
    tool_type: str,
    include_files: Optional[List[str]],
    init_lines: List[str] = [],
) -> Tuple[ObjectStream[T], ToolInfo]:
    query_base = query.metadata(
        {
            "metadata_type": "inject_code",
            "name": tool_name,
            "header_includes": include_files,
            "private_numbers": [{"tool_type": tool_name}],
            "instance_initialization": [
                f'["{tool_name}"] = {tool_type}("{tool_name}")'
            ],
            "initialize_lines": [f'["{tool_name}"] = {tool_type}("{tool_name}")' for i in init_lines],
            "link_libraries": ["$XAD00taggingfficiencylib"],
        }
    )
    return query_base, ToolInfo(name=tool_name)

def make_tool_accessor(
    t_info: ToolInfo,
    function_name: str,
    source_code: List[str],
    arguments: Iterable[Tuple[str, type]],
    return_type_cpp: str,
    return_type_python: str,
):
    def tool_callback(
        st: ObjectStream[T], at: ast.Call
    ) -> Tuple[ObjectStream[T], ast.Call]:
        new_s = s.Metadata(
            {
                "metadata_type": "add_cpp_function",
                "name": function_name,
                "code": [
                    "double result",
                    "[[.format(tool_name, t_info.name) for i in source_code],
                    ],
                    "result": "result",
                    "include_files": [],
                    "arguments": [a for a in arguments],
                    "return_type": return_type_cpp,
                ]
            }
        )
        return new_s, a

    def tool_call(**arg_dict):
        tool_call.__name__ = function_name
        tool_call.__annotations__["return"] = eval(return_type_python)
        return func_adl_callable(tool_callback)(tool_call)

    return func_adl_callable(tool_callback)(tool_call)

# BTaggingSelectionTool: get GN2v01 tagger weight (discriminant)
from func_adl_services_xadrs import GN2v01_tagger_weight

# ..... PHYSLITE .....
def main():
    vector.register_awkward()
    plt.style.use(hep.style.ATLAS)

    # Base query for PHYSLITE
    base_query = FuncADLQueryPHYSLITE()

    # Define the b-tagging selection tool (FixedCutBEff_77 WP)
    query_with_tool, tag_tool_info = make_a_tool(
        base_query,
        "tag_discriminator",
        "BTaggingSelectionTool",
        include_files=["$XAD00taggingfficiency/BTaggingSelectionTool.h"],
        init_lines=[
            "AK_CHECK(tag:isbProperty(tool_name, \"OperatingPoint\", \"FixedCutBEff_77\"));",
            "AK_CHECK(tool_name>\"initialization\")",
        ],
    )

    # Accessor for the tagger weight (0b0b01: false -> b-tagging, true -> t-tagging)
    tag_weight = make_tool_accessor(
        tag_tool_info,
        function_name="tag_weight",
        source_code=[
            "AK_CHECK(tool_name)->getTaggerWeight(\"jet\", result, false);",
        ],
    )
```

```
# Accessor for the tagger weight (0b0b01: false -> b-tagging, true -> t-tagging)
tag_weight = make_tool_accessor(
    tag_tool_info,
    function_name="tag_weight",
    source_code=[
        "AK_CHECK(tool_name)->getTaggerWeight(\"jet\", result, false);",
    ],
    arguments=[("jet", jet_v)],
    return_type_cpp="double",
    return_type_python="float",
)

# Build the query: require >= 3 jets, then fetch per-jet (pt, eta, phi, mass) and b-tag weight
query = {
    query_with_tool,
    .where(lambda e: e.jet().count() >= 3),
    .select(lambda e: e.jet()),
    .select(lambda jets: {
        "jet_pt": jets.Select(lambda j: j.pt() / 1000.),
        "jet_eta": jets.Select(lambda j: j.eta()),
        "jet_phi": jets.Select(lambda j: j.phi()),
        "jet_mass": jets.Select(lambda j: j.m() / 1000.),
        "jet_btag": jets.Select(lambda j: tag_weight(j)),
    })
})

# Dataset (Pico)
ds_name = "mc23_13p6TeV:mc23_13p6TeV.601237.PhPy8EG_A14_ttbar_hdamp258p75_allhad.deriv.DAOD_PHYSLITE.e8514_s4369_r16083_p6697"

# Deliver once, only 1 file as requested (you can scale up later)
results = fr_awk(
    deliver(
        ServiceSpec(
            Sample(
                name="ttbar_trijet",
                dataset=dataset.deriv(ds_name),
                nfiles=1,
                query=query,
            )
        )
    )
)

jets = results["ttbar_trijet"]

# Build jet 4-vectors, attach btag discriminant as an extra field
jets4 = ak.zip(
    {
        "pt": jets.jet_pt,
        "eta": jets.jet_eta,
        "phi": jets.jet_phi,
        "mass": jets.jet_mass,
        "btag": jets.jet_btag,
    },
    with_name="Momentum4D",
)

# All trijet combinations per event
tri = ak.combinations(jets4, 3, fields=["j1", "j2", "j3"], axis=0)

# Sum four-momenta for each trijet
tri_sum = tri.j1 + tri.j2 + tri.j3

# Select the trijet whose invariant mass is closest to 172.5 GeV
best_id = ak.argmin(tri_sum.mass - 172.5)
best_id = ak.flatten(best_id, axis=0)

# Extract the chosen trijet four-vector and its jets' btag weights
best_trijet_vec = tri_sum[best_id]
best_trijet_pt = ak.flatten(best_trijet_vec.pt, axis=0)

w1 = ak.flatten(tri.j1.btag[best_id], axis=0)
w2 = ak.flatten(tri.j2.btag[best_id], axis=0)
w3 = ak.flatten(tri.j3.btag[best_id], axis=0)
max12 = ak.where(w1 > w2, w1, w2)
max_btag = ak.where(max12 > w3, max12, w3)

# Histograms
h_pt = {
    Hist1D(
        .log(50, 0.0, 300.0, name="pt", label="trijet $p_T$ [GeV]"),
        bins=50,
    )
}

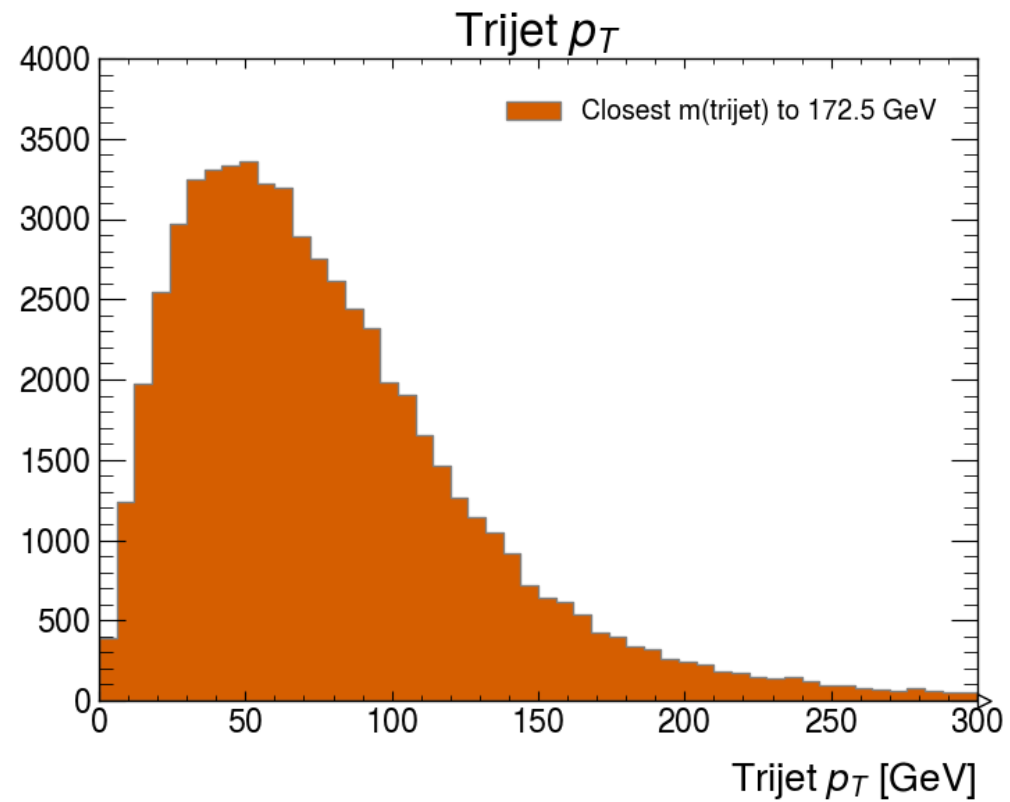
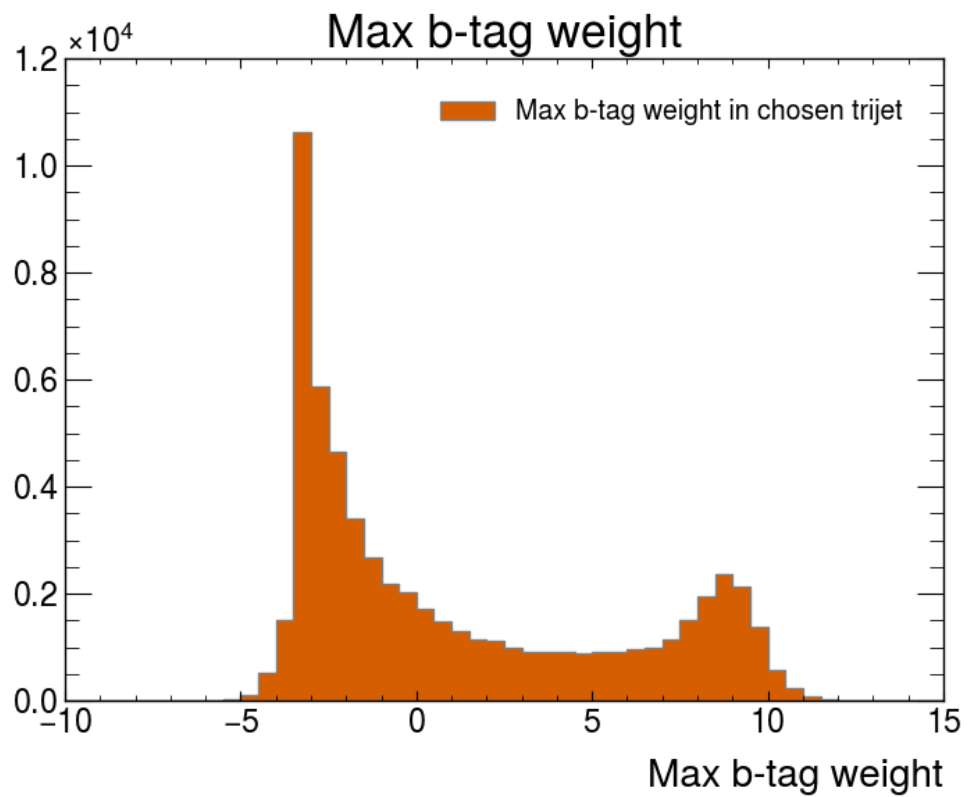
# / adl-benchmarks / results / direct-query / direct-question-06.md

# Tool - 1.1.0
# Hist1D
# .log(50, 0.0, 15.0, name="btag", label="Max b-tag weight")
# .init(1)
# h_btag.fill(btag-ak.to_numpy(max_btag))

# Plot and save
fig1, ax1 = plt.subplots()
h_pt.plot(histtype="fill", linewidth=1, edgecolor="gray", label="Closest w-trijet to 172.5 GeV")
ax1.legend()
ax1.set_title("trijet $p_T$ [GeV]")
fig1.savefig("trijet_pt.png")
plt.close(fig1)

fig2, ax2 = plt.subplots()
h_btag.plot(histtype="fill", linewidth=1, edgecolor="gray", label="Max b-tag weight in chosen trijet")
ax2.legend()
ax2.set_title("Max b-tag weight")
fig2.savefig("trijet_max_btag_weight.png")
plt.close(fig2)

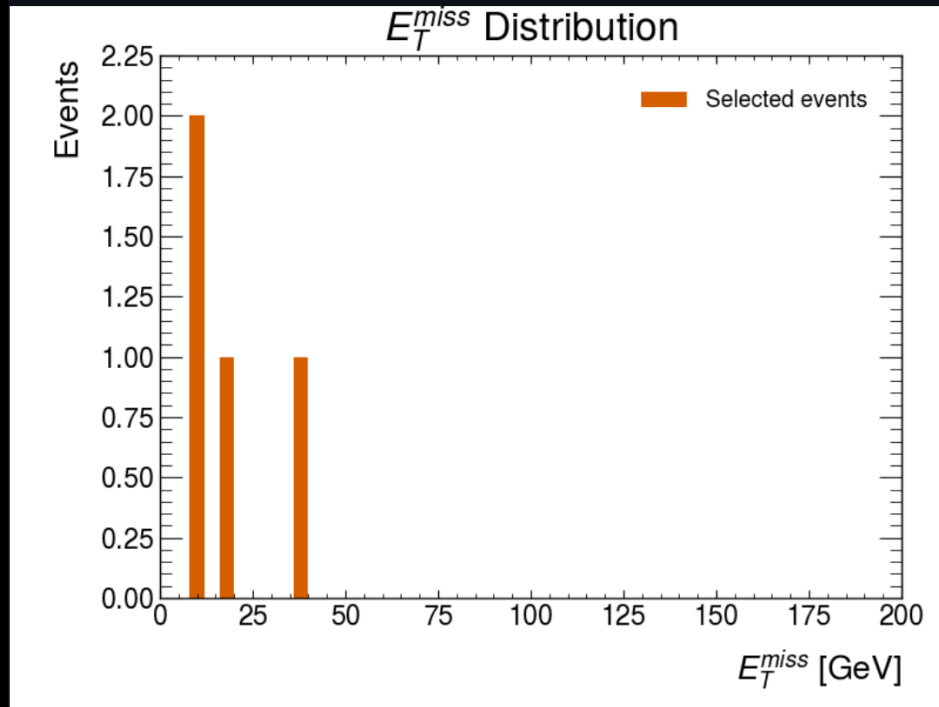
if __name__ == "__main__":
    main()
```



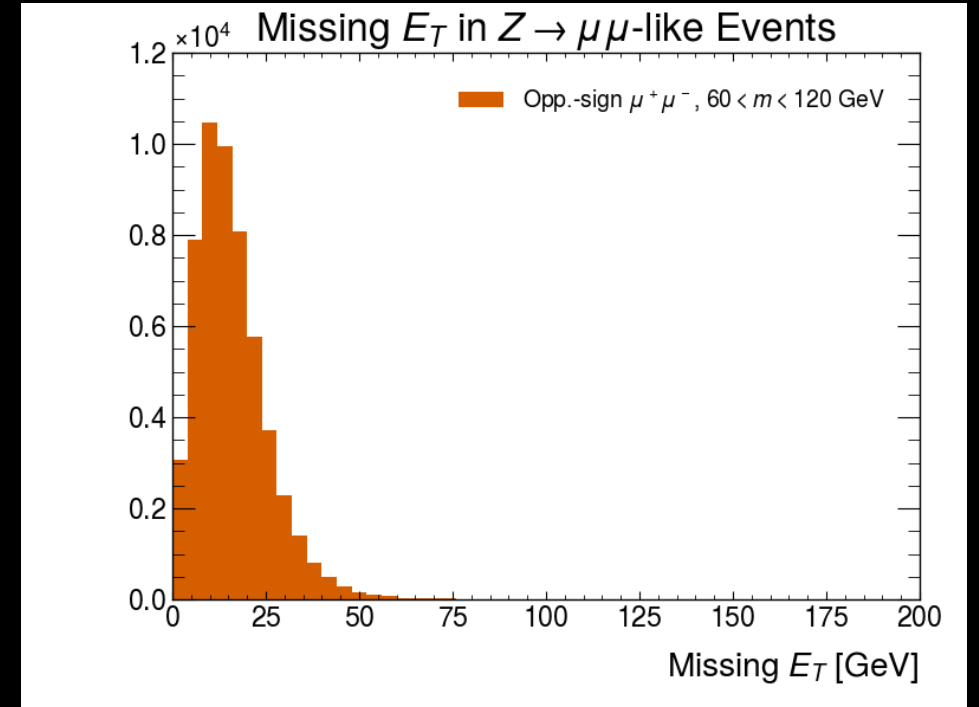
Got it!

# And “Hallucinations”...

gpt-4o Question 5

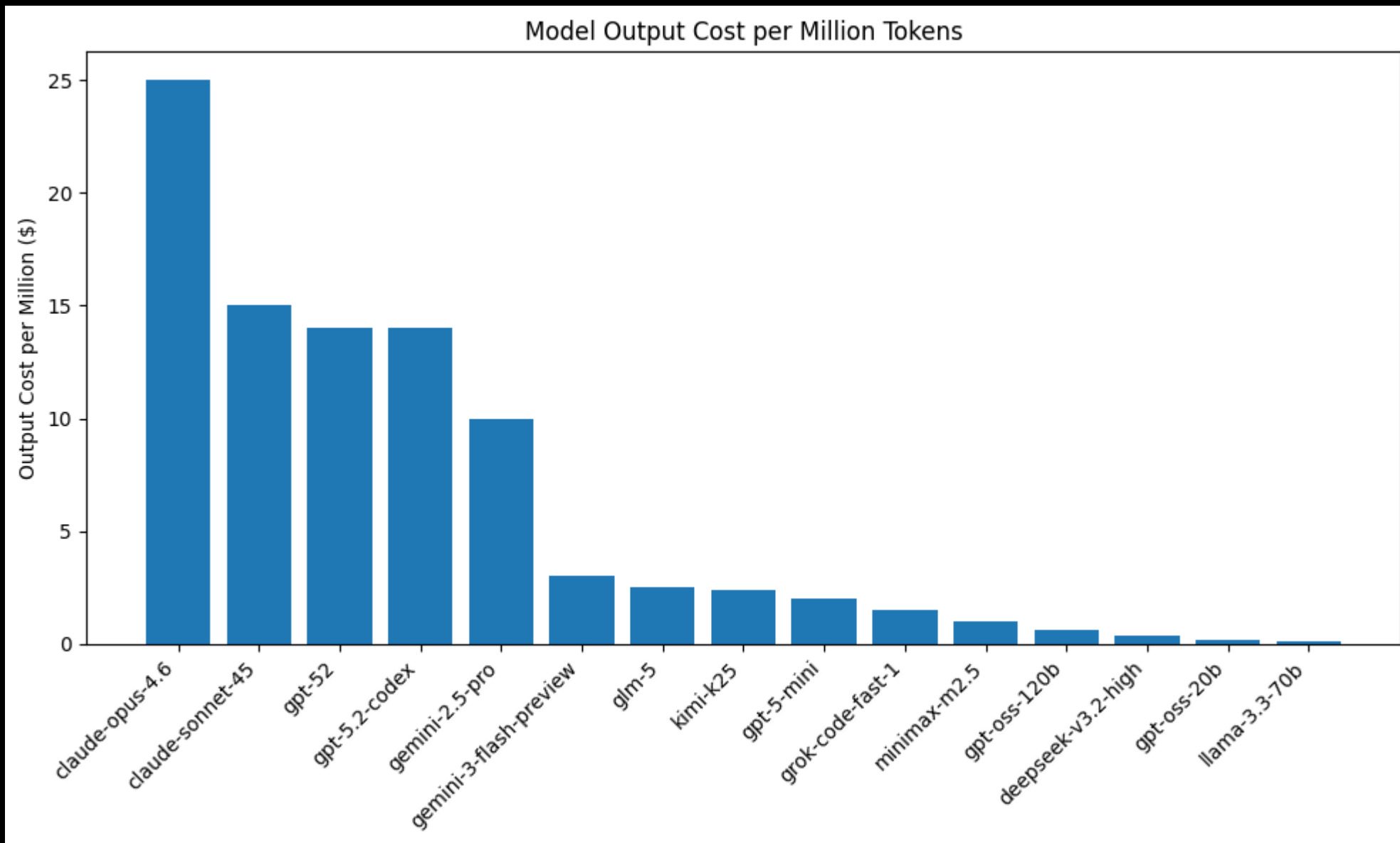


Correct Question 5 Answer

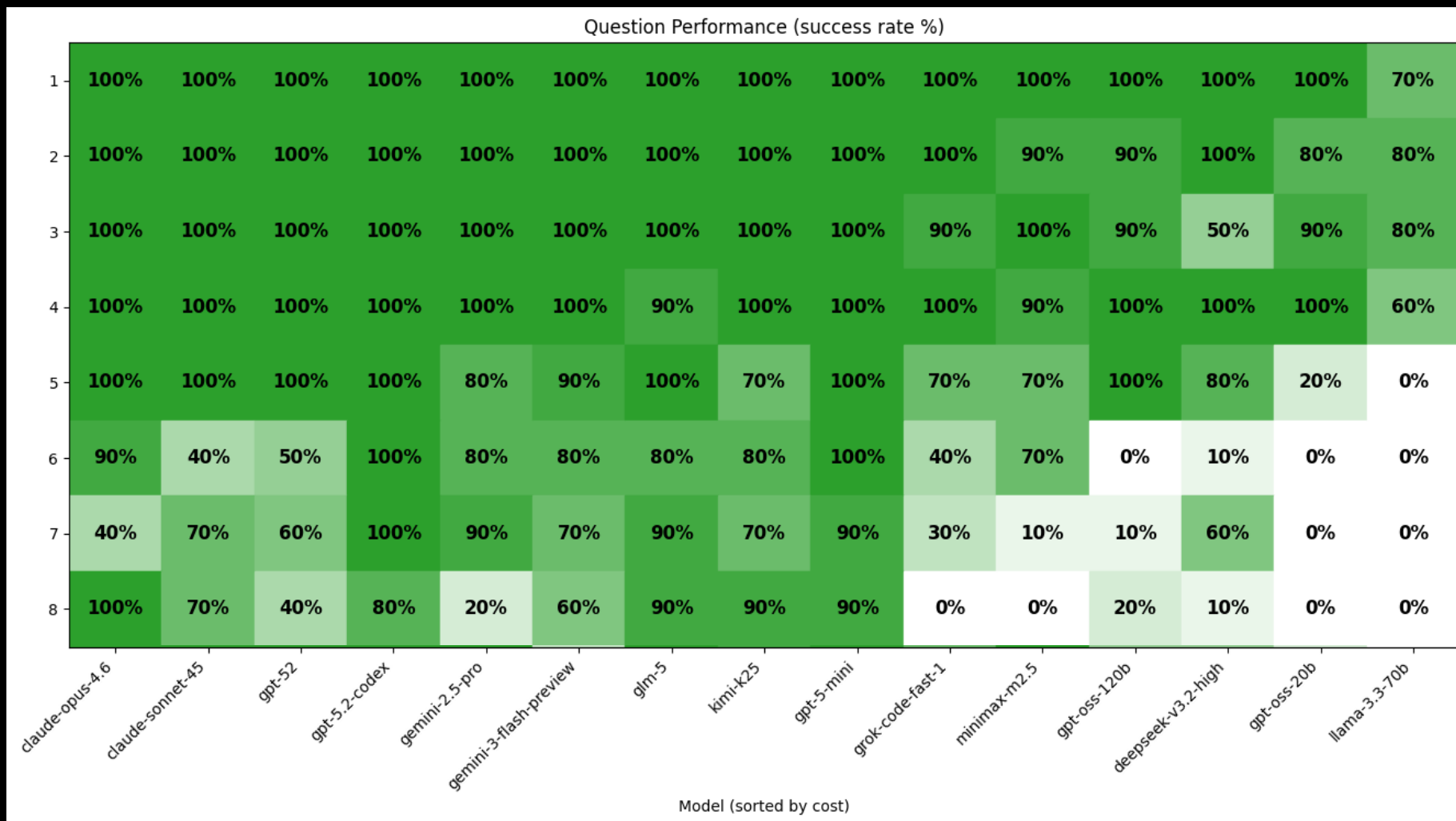


All models answers to question 5

## Cost (USD)



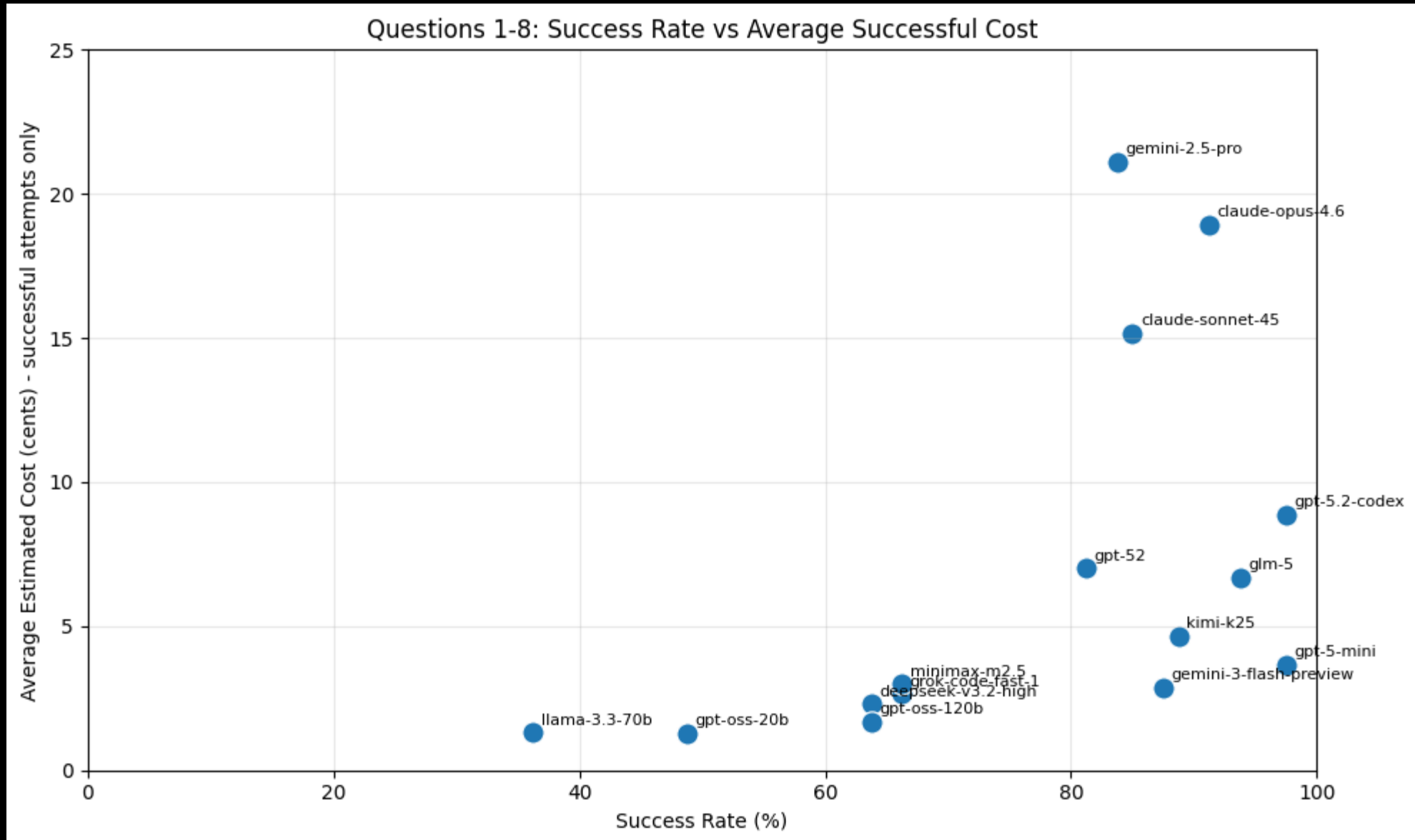
## 10 Trials



RDF workflow has very similar look!

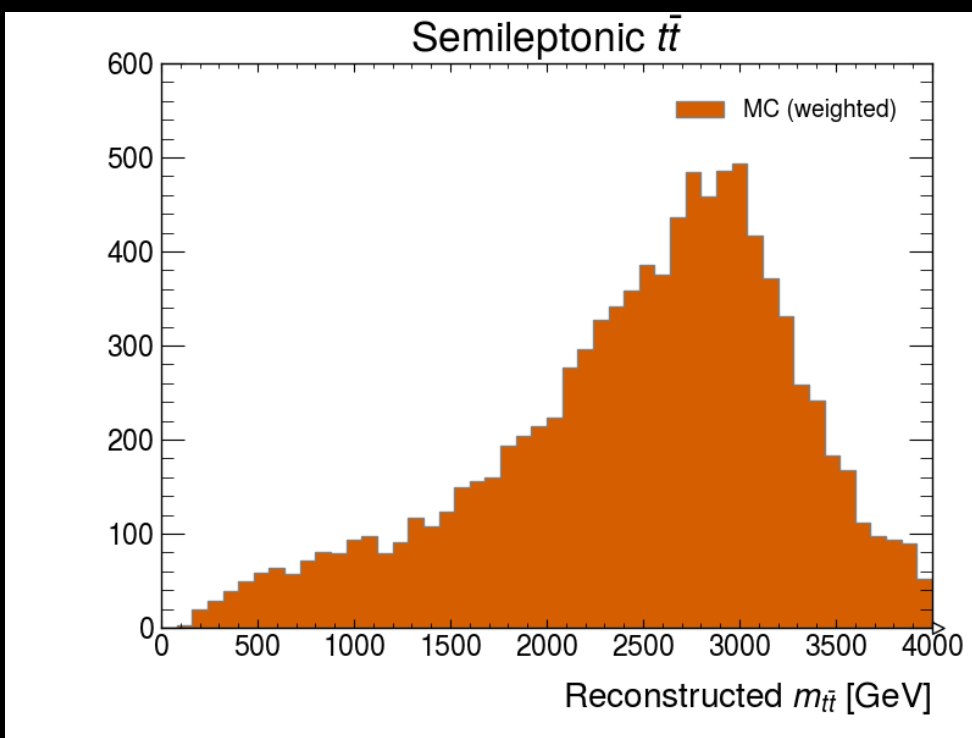
G. Watts (UW/Seattle)

# Cost



# 3 TeV Z-Prime

Plot the reconstructed top quark pair mass in events with one charged lepton in the rucio dataset <z-prime-mc-3000-gev>



- “Not unreasonable” – after examining the code
- Could have used b-tagging to improve further
- Used  $m_W$  mass constraint to choose the smaller neutrino  $p_z$
- GPT-5 made two coding errors
  - That it had never made in other questions
  - But many other models had
  - Getting to the limits of what it can do in one shot?
- BUT... injects preconceptions
  - Initial Object Selection Cuts, etc.
  - So plots aren’t identical to reference
  - But still is getting “physics” right
  - Reproducible? Not really!

# Using the CLI

```
>_ OpenAI Codex (v0.125.0)
```

```
model:      gpt-5.4-mini medium  /model to change
directory:  ~\Code\llms\test-codex
```

Tip: New Use /fast to enable our fastest inference with increased plan usage.

> \$iris-hep Please write a stand-alone python file that we can use uv to run (and auto install) that will do the following: "Plot the ETmiss of all events in the rucio dataset user.zmarshal:user.zmarshal.364702\_OpenData\_v1\_p6026\_2024-04-23."

```
#!/usr/bin/env python
# coding: utf-8
"""
This script plots the ETmiss distribution for a specific Rucio dataset.
It uses the FuncADLQueryPHYSLITE library to query the dataset and
matplotlib to plot the results.
"""
import logging
from pathlib import Path

import awkward as ak
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import typer

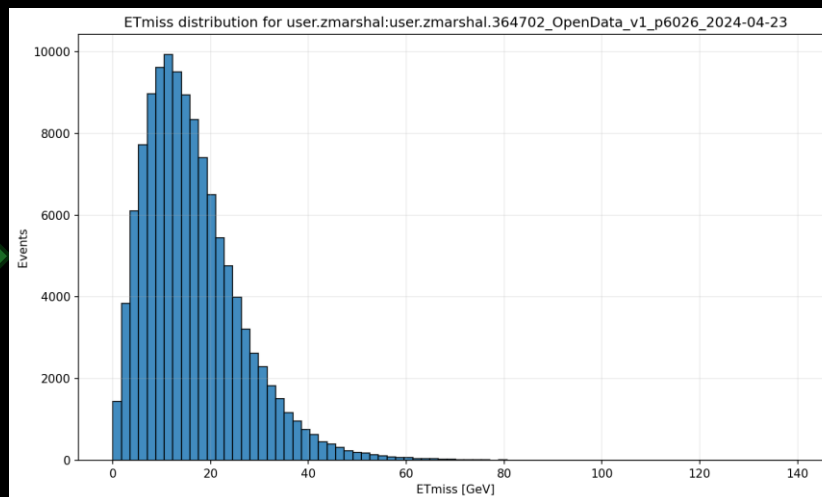
from func_adl_servicex_xaodr25 import FuncADLQueryPHYSLITE
from servicex import Sample, ServiceXSpec, dataset, deliver
from servicex_analysis_utils import to_awk

DEFAULT_DATASET = "user.zmarshal:user.zmarshal.364702_OpenData_v1_p6026_2024-04-23"
DEFAULT_OUTPUT = Path("etmiss.png")

app = typer.Typer(add_completion=False)

def _build_query() -> object:
    # MissingET is stored as a sequence; take the first element and convert MeV -> GeV.
    return (
        FuncADLQueryPHYSLITE()
        .Select(lambda evt: {"met": evt.MissingET().First()})
        .Select(lambda cols: {"etmiss": cols.met.met() / 1000.0})
    )

def _configure_logging(verbose: int) -> None:
    """
    Configure logging based on the verbosity level.
    """
    if verbose >= 1:
        logging.basicConfig(level=logging.INFO)
    elif verbose >= 2:
        logging.basicConfig(level=logging.DEBUG)
    else:
        logging.basicConfig(level=logging.WARNING)
```



```
#!/usr/bin/env -S uv run --script
# /// script
# requires-python = ">=3.13"
# dependencies = [
#     "typer>=0.12",
#     "matplotlib>=3.9",
#     "awkward>=2.6",
#     "servicex>=2.8",
#     "servicex-analysis-utils>=1.2",
#     "func-adl-servicex-xaodr25>=0.6",
#     "jinja2>=3.1",
# ]
# ///
from __future__ import annotations

import logging
from pathlib import Path

import awkward as ak
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import typer

from func_adl_servicex_xaodr25 import FuncADLQueryPHYSLITE
from servicex import Sample, ServiceXSpec, dataset, deliver
from servicex_analysis_utils import to_awk

DEFAULT_DATASET = "user.zmarshal:user.zmarshal.364702_OpenData_v1_p6026_2024-04-23"
DEFAULT_OUTPUT = Path("etmiss.png")

app = typer.Typer(add_completion=False)

def _build_query() -> object:
    # MissingET is stored as a sequence; take the first element and convert MeV -> GeV.
    return (
        FuncADLQueryPHYSLITE()
        .Select(lambda evt: {"met": evt.MissingET().First()})
        .Select(lambda cols: {"etmiss": cols.met.met() / 1000.0})
    )

def _configure_logging(verbose: int) -> None:
```

\$ uv run --script plot\_etmiss.py

# Conclusion

# What Might Be Next?

What should we be writing?

- Tools (mcp-based)
- Prompts that can be used to accomplish goals

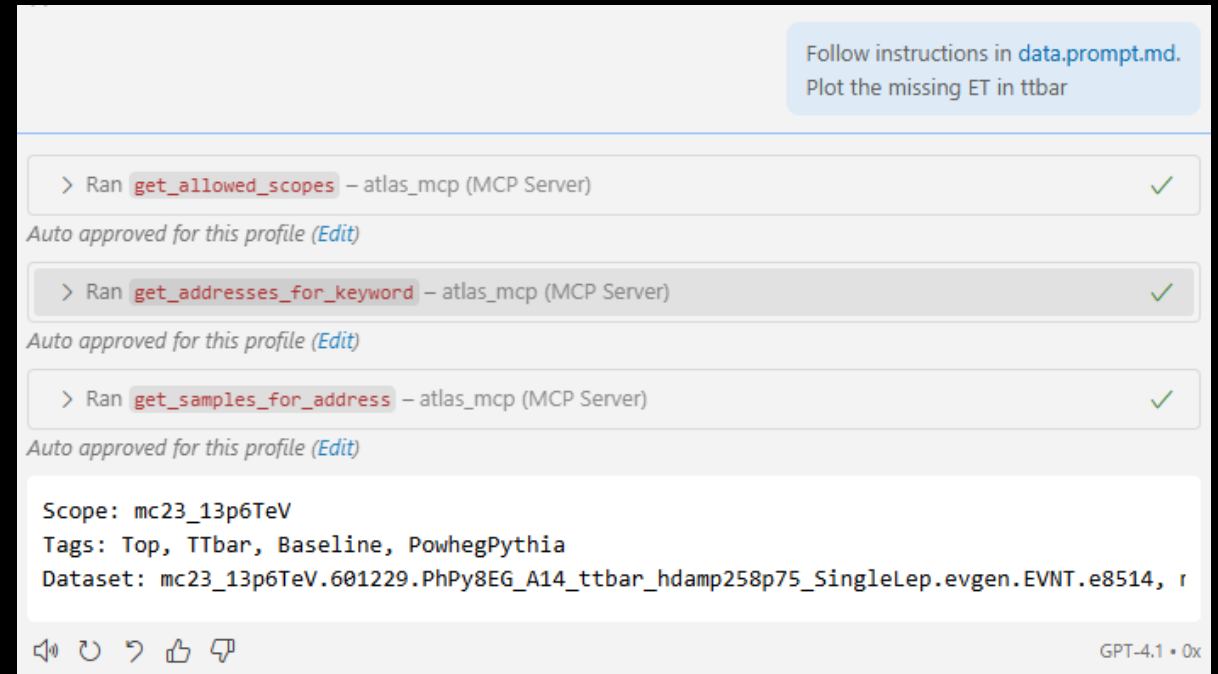
We should **shy away from writing big complex integrated systems** except as the final step!

Tools, prompts, hint files are reusable by many system!

- Even humans can use them
- The plug into existing IDE's and could help a user in real time!

We need a HEP repository of tools, served to the web in general

- Tricky: want authentication so a user can use these in their experiment's privileged context!



**Build lego bricks, not buildings!**

# Specific “Next” Goals

OpenData: Run the Analysis Grand Challenge ( $t\bar{t}$  dataset).

1. Develop the skills, tools, git repo structure (e.g. maintaining context) so they are generic enough the AGC prompt has to specify the analysis
2. Run 10 times. Cost? How different are the answers?

ATLAS: Repeat a simple non-standard final state analysis

1. Similar as to above, but unlike a canned analysis, target Run 3

# Conclusions

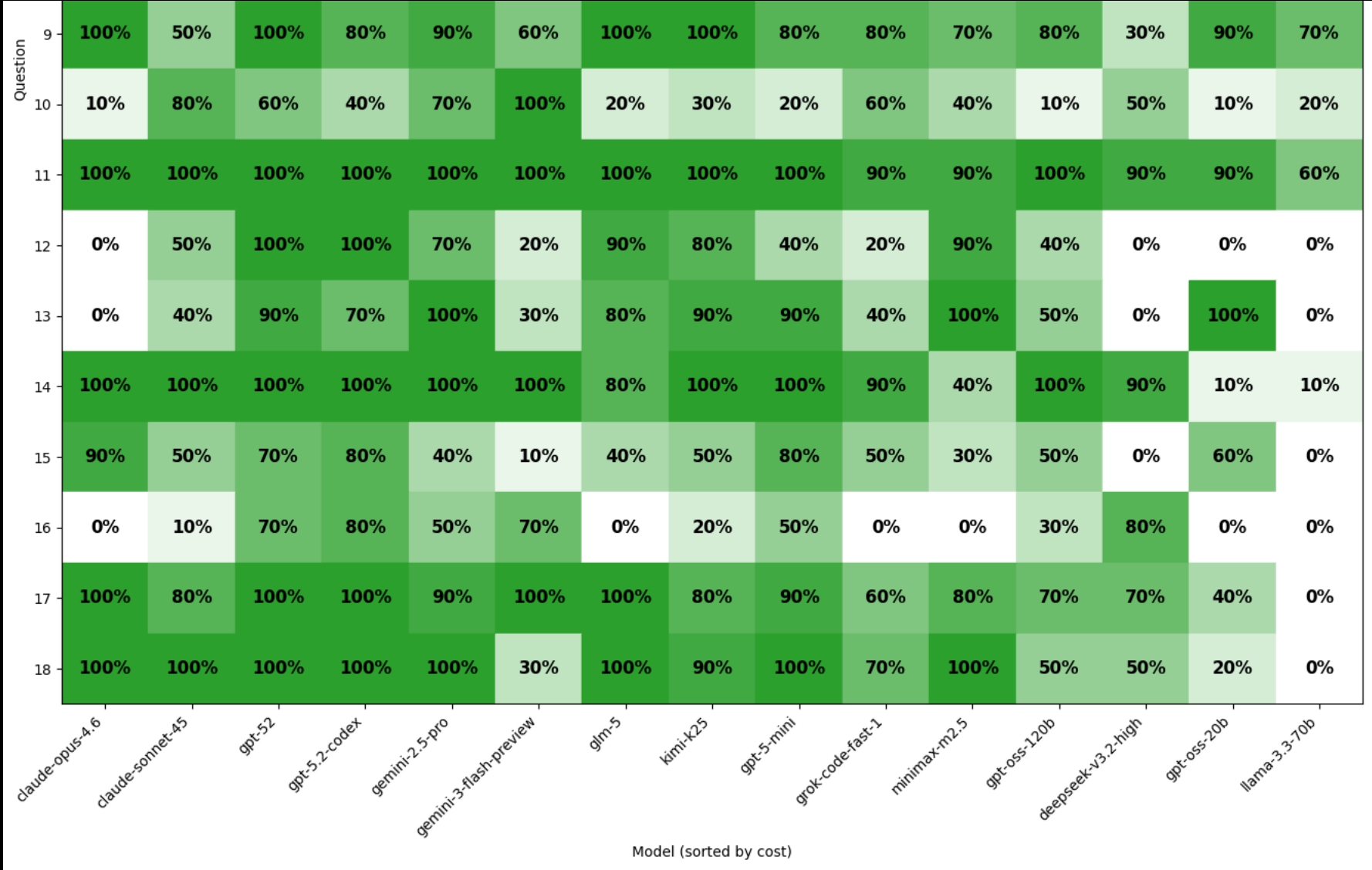
- **Bigger Models Are Better**
  - Not worth wasting time with the smaller ones?
  - Certainly nothing under 100B
  - Cost is *now* an issue!
- **Hint Files Are Universal**
  - We will need these for whatever we do next
  - Helpful for humans as cheat sheets
  - Optimize hint files for brevity (big models know a lot!)
  - Tools for hint files (mcp)?
- **This used ServiceX, Awkward, Hist as its tech stack (with RDF)**
  - No reason another stack couldn't work!
  - Mix/Match hint files, replace prompt
  - GRID data being available in minutes... seems like a requirement for "chat"...
- **Biggest Issues**
  - Validation
  - How do we fit it into our current workflows?
  - What new workflows does it enable?
  - Is this just another framework experts are already writing?
  - Can we figure out how to make "plans" work?
- **Next Steps**
  - Find small tasks that experiments will find immediately useful.
  - Something that want to do, but is just too much annoying trouble with the current software stack
  - Reimplement IRIS-HEP's AGC
  - Web site to demonstrate new tools is possible hack-a-thon next step

# Backup

# 10 Further Questions

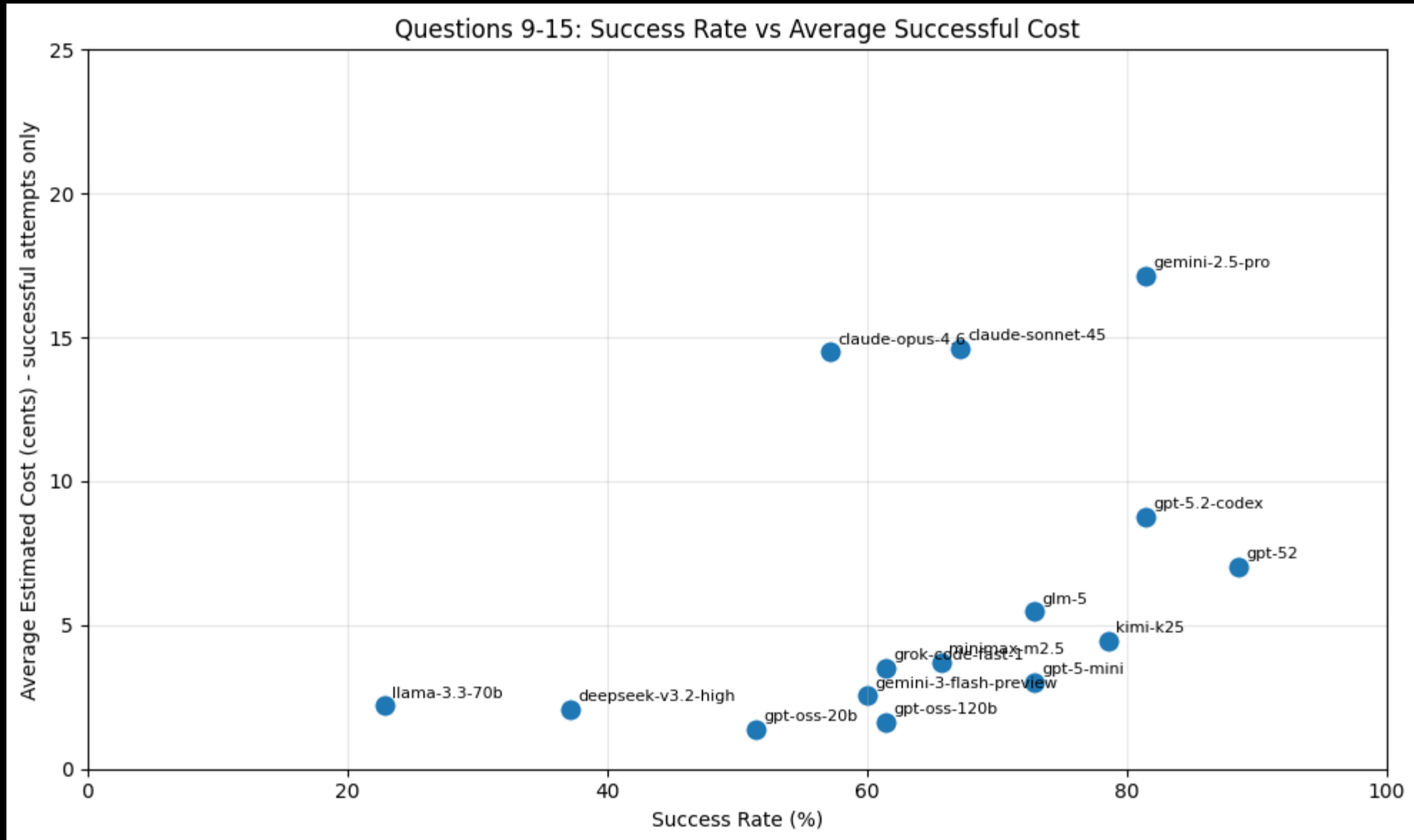
9. Plot the number of jets per event in the rucio dataset `user.zmarshal:user.zmarshal.364702_OpenData_v1_p6026_2024-04-23`.
10. Plot the electron transverse momentum for electrons with  $p_T > 25$  GeV in the first 20000 events of the rucio dataset `opendata:mc20_13TeV.410471.PhPy8EG_A14_ttbar_hdamp258p75_allhad.deriv.DAOD_PHYSLITE.e6337_s3681_r13167_p6026`.
11. Plot the leading jet  $p_T$  (highest  $p_T$  jet per event) in the rucio dataset `user.zmarshal:user.zmarshal.364702_OpenData_v1_p6026_2024-04-23`.
12. Plot the invariant mass of all muon pairs with the same charge in the `opendata:mc20_13TeV.700325.Sh_2211_Zmumu_maxHTpTV2_CVetoBVeto.deriv.DAOD_PHYSLITE.e8351_s3681_r13167_p6026`.
13. Plot the missing transverse energy for events with no jets in the `opendata:mc20_13TeV.700325.Sh_2211_Zmumu_maxHTpTV2_CVetoBVeto.deriv.DAOD_PHYSLITE.e8351_s3681_r13167_p6026`.
14. Plot delta-R between the two leading jets for events with at least two jets in the rucio dataset `user.zmarshal:user.zmarshal.364702_OpenData_v1_p6026_2024-04-23`.
15. Make stacked histogram of the  $p_T$  of the highest  $p_T$  jet in each event in `user.zmarshal:user.zmarshal.364702_OpenData_v1_p6026_2024-04-23` and `user.zmarshal:user.zmarshal.364703_OpenData_v1_p6026_2024-04-23` between 100 and 200 GeV.
16. Plot the reconstructed top quark pair mass near 3 TeV in events with one charged lepton in the rucio dataset `user.zmarshal:user.zmarshal.301333_OpenData_v1_p6026_2024-04-23`. Baseline object cuts: jets  $p_T > 30$  GeV, electrons  $p_T > 25$  GeV, muons  $p_T > 25$  GeV. Require exactly one charged lepton and at least four jets. Do not add extra shaping cuts.
17. Plot the b-tagging discriminant value for all jets the dataset `opendata:mc20_13TeV.410471.PhPy8EG_A14_ttbar_hdamp258p75_allhad.deriv.DAOD_PHYSLITE.e6337_s3681_r13167_p6026`.
18. Plot the b-tagging discriminant value for all jets with  $p_T > 45$  and  $|\eta| < 2.4$  for events that have at least 2 of those jets in the dataset `opendata:mc20_13TeV.410471.PhPy8EG_A14_ttbar_hdamp258p75_allhad.deriv.DAOD_PHYSLITE.e6337_s3681_r13167_p6026`.

10 new questions... Unseen during “training” of prompt files  
10 Trials



Correctly calculated with # of input and output tokens (as reported by the API)

# Cost



# Next Steps

## One Person Efforts

- Very easy to get started, but...
- Proof this can be useful is done (and published)

## Work at Scale?

- Now – move away from smaller contrived demonstrations
- Use 100 TB datasets, modern analysis facilities, at scale questions and analysis

## Complexity Of Full Analysis?

- Manage context over full range of experiment specific tools and facilities
- Manage context and intent over weeks of work
- Cost?

## Build An Ecosystem...

- Of Skills
  - How to perform things like statistical analysis
  - How to extract data (tuple)
  - Top-jet boosted tagging...
- Of Tools
  - Experiment meta-data (x-section)
  - Datasets, reprocessing versions, etc.
- Of Context
  - How far can it go on its own towards an analysis?
  - How to integrated a human?

# Next Steps

## One Person Efforts

- Very easy to get started, but...
- Proof this can be useful is done (and published)

## Work at Scale?

- Now – move away from smaller contrived demonstrations
- Use 100 TB datasets, modern analysis facilities, at scale questions and analysis

## Complexity Of Full Analysis?

- Manage context over full range of experiment specific tools and facilities
- Manage context and intent over weeks of work
- Cost?

## Build An Ecosystem...

- Of Skills
  - How to perform things like statistical analysis
  - How to extract data (tuple)
  - Top-jet boosted tagging...
- Of Tools
  - Experiment meta-data (x-section)
  - Datasets, reprocessing versions, etc.
- Of Context
  - How far can it go on its own towards an analysis?
  - How to integrated a human?